

Programación II

Desarrollo en Java

Clase N° 15:

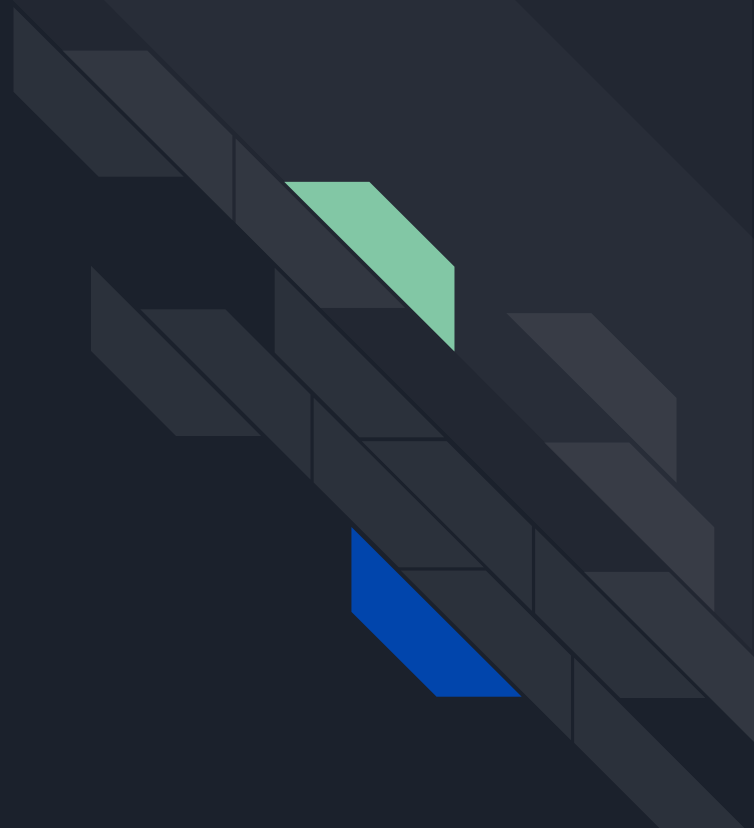
Manejo de errores / Excepciones

Profesora Carolina Archuby



TEMAS A DESARROLLAR

- Concepto de Excepción.
- Tipos de excepciones.
- ¿Por qué utilizar excepciones?
- Excepciones checked o comprobadas.
- Excepciones unchecked o no comprobadas
- Lanzamiento de excepciones.
- Transferencia de control.
- Cláusula throws.
- Terminología utilizada para trabajar con excepciones.
- Creando nuestras propias excepciones.
- Malas prácticas vs buenas prácticas.



EXCEPCIONES

- Durante la ejecución del programa pueden producirse **errores** o situaciones excepcionales que interrumpen el flujo normal del programa (por ejemplo: *leer un archivo que no existe, acceder al valor N de una colección que tiene menos de N elementos, enviar o recibir información por red mientras se produce una pérdida de conectividad, etc*).
- Generalmente, ante estos errores, el programa se cierra. El **manejo de excepciones** es el **mecanismo** previsto por Java para permitir **tratar** estos **errores**: capturarlos, recuperarnos, mostrar un mensaje y decidir si continuamos o no.
- En Java, **las excepciones son objetos** que encapsulan la información del error ocurrido. **Convierten los errores que un método puede arrojar en una parte explícita del contrato del método.**

VENTAJAS DEL USO DE EXCEPCIONES

Al obligar a los desarrolladores a considerar situaciones de error a través del manejo explícito de excepciones, Java ayuda a **prevenir fallos no controlados en el software, llevando a aplicaciones más estables y confiables.**

Las excepciones **permiten que el software responda de manera adecuada ante fallos.**

VENTAJAS DEL USO DE EXCEPCIONES

Separa el código que gestiona los errores del código principal del programa

Nos permite GESTIONAR el error y continuar con la ejecución del programa

Agrupar y diferenciar entre diferentes tipos de errores

Propagar errores hacia arriba en la pila de llamadas (StackTrace)

¿Por qué consideramos las condiciones del error?

Ante un problema, el programa puede:

- **Terminar** inmediatamente la ejecución del programa



- **Ignorar** el error y continuar.



- Establecer una indicación, señal o retorno que (esperemos) alguien tome y actúe en consecuencia.



¡Cuidado!

“El **manejo de errores** usando excepciones **no evita errores**,
sólo permite su **detección** y su **posible reparación**.”



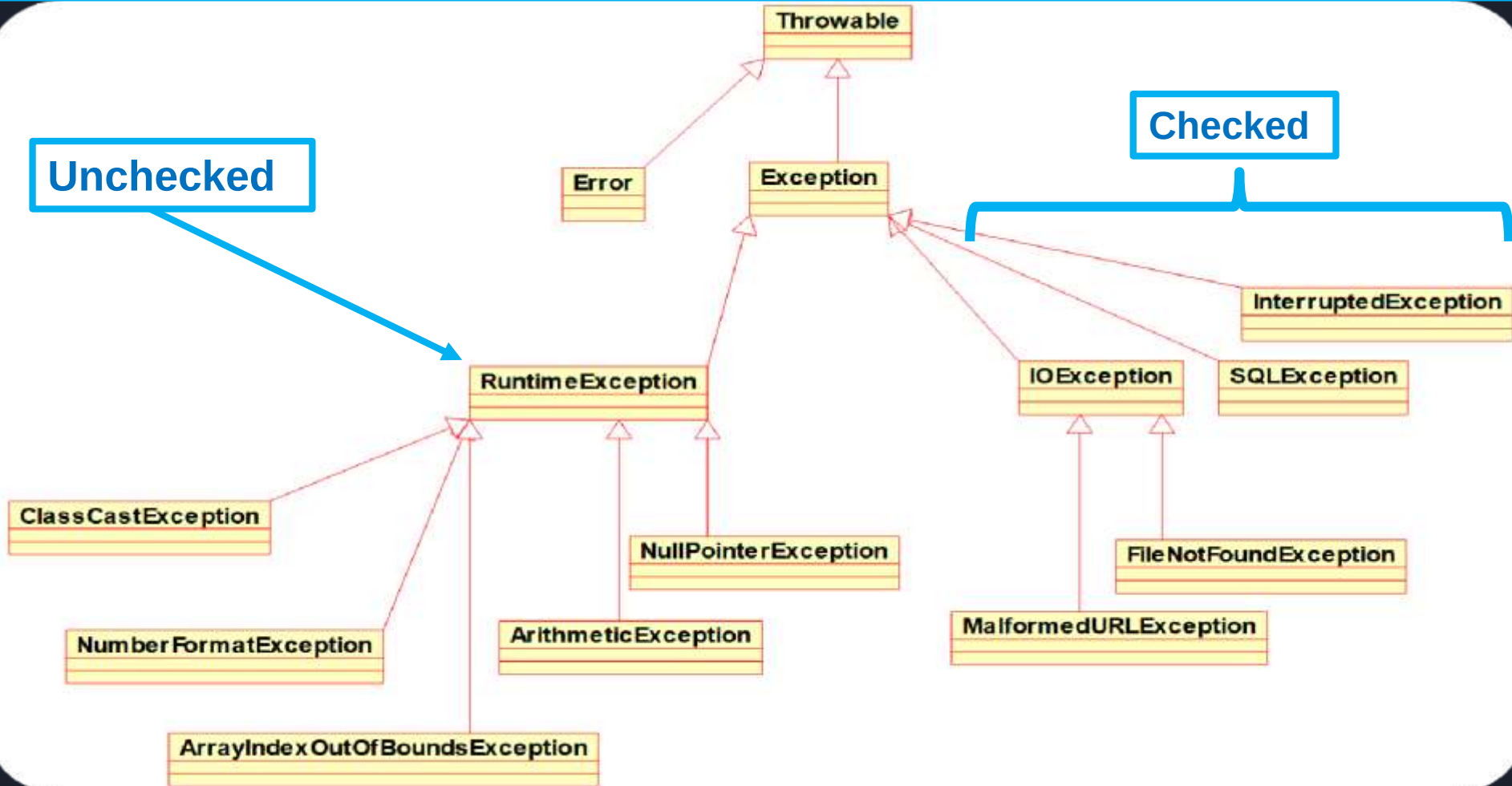
TIPOS DE EXCEPCIONES

- Las excepciones también son OBJETOS.
- Todos los tipos de excepción deben extender de la **clase base Throwable** (la superclase de todas las clases de errores y excepciones en Java) o de alguna de sus **subclases**.
- De Throwable nacen dos grandes ramas: Error y Exception.

TIPOS DE EXCEPCIONES

Unchecked

Checked



TIPOS DE EXCEPCIONES

1) ERROR: Representa aquellos **errores** que son **irrecuperables**, errores de una magnitud tal que una aplicación nunca debería intentar realizar nada con ellos. Generalmente están relacionados con problemas del entorno de ejecución de la aplicación, como quedarse sin memoria (OutOfMemoryError), errores de la JVM, desbordamiento de buffer.

2) EXCEPTION Representa aquellos **errores que sí deberíamos tratar de gestionar**. De Exception derivan clases de las que se instancian objetos para ser lanzados y luego capturados por los catch Por convenio, los nuevos tipos de excepción extienden a Exception.

De Exception nacen múltiples ramas, que a su vez pueden clasificarse en excepciones checked y unchecked.

EXCEPCIONES CHECKED (COMPROBADAS)

=> Una excepción de tipo checked **representa un error del cual técnicamente podemos recuperarnos.**

=> Son todas las **SITUACIONES** que son totalmente **AJENAS AL PROPIO CÓDIGO**, por ejemplo: fallo de una operación de lectura/escritura.

=> El compilador obliga a que **DEBAN SER MANEJADAS EXPLÍCITAMENTE POR EL PROGRAMA** (capturadas o relanzadas por el propio método para que quien lo invoque sí las capture). **Sino no compilará.**

EXCEPCIONES UNCHECKED (NO COMPROBADAS)

=> Representan ERRORES DE PROGRAMACIÓN.

=> No necesitan ser declaradas en un método o constructor mediante una cláusula throws, y EL COMPILADOR NO REQUIERE QUE SEAN CAPTURADAS

* Por ejemplo: intentar leer de un array de N elementos un elemento que se encuentra en una posición mayor que N.

```
Int [ ] numerosPrimos = {1, 3, 5, 7, 9, 11, 13, 17, 19, 23};
```

```
System.out.println (numerosPrimos[10]); → ArrayIndexOutOfBoundsException.
```

* Otro error puede ser el de dividir algún número por 0, esto nos arrojaría un `ArithmeticException`, o intentar manejar un objeto nulo (`NullPointerException`)

RuntimeException

=> Pertenecen a la clase EXCEPTION

=> Son excepciones que **se propagan automáticamente sin necesidad de especificarlas en la cabecera de los métodos.**

=> Por ejemplo: En la división entre 0 o índice fuera de rango sería tedioso especificar en todos los métodos que se puede propagar dicha excepción, por ello, Java las propaga sin tener que especificar ese hecho.

Mecanismo del manejo de excepciones en Java

- try** • es un bloque para detectar excepciones,
- catch** • es un manejador para **capturar** excepciones de los bloques try,
- throw** • es una expresión para **lanzar** (throw) una excepción en el momento que detecta el error.
- throws** • indica las excepciones que puede elevar un método.
- finally** • es un bloque opcional situado después de los catch de un try.

Mecanismo del manejo de excepciones en Java: try-catch

```
try {
```

[método o bloque que lanza la excepción]

```
} catch (TipoDeExcepcion variable) {
```

[acá manejamos el error]

```
} finally {
```

[bloque opcional, siempre se ejecuta]

```
}
```

Bloque try:

=> El bloque try **es donde se coloca el código que puede causar una excepción.**

=> Este bloque intenta ejecutar dentro de él el código que es susceptible a errores (como operaciones de entrada/salida, parseo de números, manejo de archivos, etc.). Si ocurre un error dentro del bloque try, la ejecución del código dentro de este bloque se detiene inmediatamente, y el control se pasa al bloque catch correspondiente que puede manejar ese tipo específico de excepción.

Bloque catch:

=> El bloque **catch captura y maneja las excepciones especificadas**. Es usado para definir una respuesta a diversas situaciones de error, permitiendo que el programa continúe su ejecución de una manera controlada o que proporcione una salida de error más comprensible para el usuario.

=> Puede haber múltiples bloques catch asociados a un solo try, permitiendo manejar diferentes tipos de excepciones de manera específica. Cada catch es diseñado para capturar un tipo de excepción en particular y contiene el código que determina cómo responder a esa situación.

Ejemplo de múltiples catch:

```
try {  
    String valor = JOptionPane.showInputDialog(null, "Ingresar un entero:");  
  
    // Un valor no numérico lanzará un NumberFormatException  
    int divisor = Integer.parseInt(valor);  
  
    // Si la división es 0, esto resultará un ArithmeticException  
    System.out.println(10 / divisor);  
  
} catch (NumberFormatException nfe) {  
    [ acá manejamos el error NumberFormatException...]  
} catch (ArithmeticException ae) {  
    [ acá manejamos el error ArithmeticException...]  
}
```

MANEJO DE EXCEPCIONES EN JAVA: LANZAMIENTO

=> Las excepciones se lanzan utilizando la palabra reservada THROW:
throw AException

=> AException debe ser un **objeto Throwable**, ya sea una excepción predefinida en Java o una excepción personalizada creada por el programador.

=> Las excepciones son objetos, por lo tanto se debe crear una instancia antes de lanzarse.

=> Las excepciones deben lanzarse en situaciones donde el método no puede completar su tarea de manera normal debido a circunstancias excepcionales. Si se lanza la excepción, no se regresa al flujo normal del programa.

MANEJO DE EXCEPCIONES EN JAVA: LANZAMIENTO

Ejemplo: Si la cantidad a retirar es mayor que el saldo disponible, se lanza la excepción `SaldoInsuficienteException` con un mensaje específico:

```
public void retirar(double monto) throws SaldoInsuficienteException {  
    if (monto > saldo) {  
        throw new SaldoInsuficienteException("Saldo insuficiente para retirar $" + monto);  
    }  
    saldo -= monto;  
    System.out.println("Retiro exitoso. Saldo restante: $" + saldo);  
}
```

MANEJO DE EXCEPCIONES EN JAVA: LANZAMIENTO

=> Cuando se lanza una excepción, lo que se hace es lanzar una instancia de `Exception` o de una clase derivada.

=> Esta clase tiene dos constructores y dos métodos importantes:

```
Exception e = new Exception();
```

```
String mensaje = "algún mensaje de error";
```

```
Exception e = new Exception (mensaje);
```

```
String mensaje = e.getMessage();
```

```
e.printStackTrace();
```

Manejo de excepciones en Java: Transferencia de control

=> Las sentencias dentro de la cláusula try se ejecutan hasta que se lance una excepción o hasta que finalicen con éxito las instrucciones del try.

=> Si se lanza una excepción, las acciones que hubiera detrás del punto donde se produjo la excepción no tienen lugar, y pasarán a examinarse **sucesivamente** las sentencias contenidas dentro de cada cláusula **catch** (en orden), y por último el bloque **finally**.

Manejo de excepciones en Java: Transferencia de control

Ejemplo:

```
public static void main(String[] args) {  
    FileWriter fichero = null;  
    try {  
        fichero = new FileWriter(PATH_ARCHIVO);  
        fichero.write("Escribiendo...")  
    } catch (IOException e) {  
        e.printStackTrace();  
    } finally {  
        if (fichero != null) {  
            fichero.close();  
        }  
    }  
}
```

Manejo de excepciones en Java: Bloque Finally

=> Contiene código que debe ejecutarse después de que los bloques try y catch hayan terminado su ejecución, independientemente de si se lanzó una excepción o no.

=> Generalmente se usa para limpiar el estado interno o para liberar recursos (como cerrar archivos o conexiones a bases de datos), independientemente de si las operaciones fueron exitosas o no.

Manejo de excepciones en Java: Propagación

=> Los métodos deben:

- * capturar todas las excepciones chequeadas que puedan ser lanzadas dentro de su ámbito,
- * o propagarlas hacia arriba en la pila de llamadas hasta que se encuentren con un bloque catch adecuado para manejarlas.

=> Esto es especialmente útil en aplicaciones grandes, donde un error en un nivel bajo puede necesitar ser tratado en un nivel más alto de manera específica. La capacidad de propagar excepciones permite a los desarrolladores decidir si un método debe manejar un error directamente o pasarlo a su llamador para que lo maneje, basándose en el contexto de la aplicación.

Manejo de excepciones en Java: Propagación

=> Si al ocurrir un **error** está activa una porción de código denominado **manejador de excepción**, entonces el flujo de control se transfiere al **manejador**.

=> Si no existe un manejador para la excepción, ésta se **propaga** al método que invoca.

=> Si en éste tampoco se capta, la excepción se propaga al que a su vez le llamó.

=> Si llega al método por el que empieza la ejecución, es decir, **main()**, y tampoco es captada, la ejecución termina.

Manejo de excepciones en Java: Throws

- => Throws es la palabra reservada que se utiliza en la firma de un método para declarar una o varias excepciones checked que puede lanzar el método. Es un aviso de que en el cuerpo del método hay por lo menos una sentencia que lanza una excepción checked.
- => Se pueden declarar varias, separadas por comas.
- => RuntimeException y Error son las únicas excepciones que no hace falta incluir en las cláusulas throws.
- => Hay muchos ejemplos (por ejemplo, los métodos de lectura de archivos). El IDE nos avisará del mismo para que lo encerremos en un bloque try-catch.

Manejo de excepciones en Java: Throws

Ejemplo:

```
PUBLIC CLASS CLIENTEREPOSITORIO {  
    PUBLIC CLIENTE PORID(INT ID) THROWS EXCEPTION {  
        [...]  
    }  
}
```

Manejo de excepciones en Java: Throws

Si se invoca a un método que tiene una excepción checked en su cláusula throws, existen tres opciones:

1) Capturar la excepción y gestionarla.

```
public void abrirArchivo(String path) throws IOException {  
    ...  
}  
  
public void leerArchivo(String path) {  
    try {  
        abrirArchivo(path);  
    } catch(IOException e) {  
        System.out.println("Se produjo un error al abrir el archivo").  
    }  
}
```

Si se invoca a un método que tiene una excepción checked en su cláusula throws, existen tres opciones:

2) Capturar la excepción y transformarla en una de nuestras excepciones.

```
public void abrirArchivo(String path) throws IOException {  
    ...  
}
```

```
public void leerArchivo(String path) {  
    try {  
        abrirArchivo(path);  
    } catch(IOException e) {  
        throw new ArchivoNoExisteException(e);  
    }  
}
```

Si se invoca a un método que tiene una excepción checked en su cláusula throws, existen tres opciones:

3) Declarar la excepción en la cláusula throws y hacer que otro método la gestione.

```
public void abrirArchivo(String path) throws IOException {  
    ...  
}  
  
public void leerArchivo(String path) throws IOException {  
    abrirArchivo(path);  
    ...  
}
```

Manejo de excepciones en Java: Throws

=> No se permite que los **métodos redefinidos** declaren más excepciones checked en la cláusula throws que las que declara el método.

=> Se pueden lanzar **subtipos** de las excepciones declaradas, ya que podrán ser capturadas en el bloque catch correspondiente a su supertipo.

CREANDO NUESTRAS PROPIAS EXCEPCIONES

=> Además de las clases de excepciones que define Java, se pueden definir nuevas excepciones personalizadas para que las aplicaciones tengan **control específico** de errores, definiendo acciones concretas ante errores particulares sin tener que depender de excepciones más genéricas que podrían no ser tan descriptivas sobre la naturaleza del error.

CREANDO NUESTRAS PROPIAS EXCEPCIONES

=> Aquellas que se definan, tienen que heredar de la clase base `Exception`.

* Ejemplo:

```
public class MiExcepcion extends Exception {  
    ...  
}
```

=> Normalmente, se definen varios constructores para proporcionar flexibilidad en cómo se puede lanzar la excepción, pasando mensajes de error o causas

=> Cada excepción en Java puede encapsular información sobre el contexto en el que ocurrió el error, incluyendo un mensaje descriptivo y un rastro de pila (stack trace) que indica la secuencia de llamadas que llevó al error. Esto es crucial para depurar y resolver problemas en el software.

1ro: Creamos nuestra propia excepción

```
public class WrongPasswordException extends Exception{

    public WrongPasswordException (String message) {
        super (message);
    }

    @Override
    public String getMessage () {
        return "Ocurrio un error de validacion con su contrasenia: "+super.getMessage();
    }
}
```

2do: Creamos el método que puede llegar a lanzar esa excepción

```
public class Authentication {  
    private String username;  
    private String password;
```

```
    public Authentication(String username, String password) {  
        this.username = username;  
        this.password = password;  
    }
```

```
    public boolean login(String password) throws WrongPasswordException {  
        if (!password.equals(this.password)) {  
            throw new WrongPasswordException("Contraseña incorrecta.");  
        }  
        return true;  
    }  
}
```

```
public static void main(String[] args) {  
    Scanner teclado = new Scanner (System.in) ;  
    boolean flag = false;  
    Authentication user = new Authentication( username: "usuario123", password: "1234");  
    System.out.println ("Ingrese su contrasenia");  
    String password = teclado.nextLine();  
    try {  
        flag = user.login (password) ;  
    } catch (WrongPasswordException e) {  
        System.out.println (e.getMessage() );  
    }  
    catch (Exception e) {  
        System.out.println (e.getMessage());  
    }  
}
```

3ro: Envolveremos la llamada al método que contiene el posible error a manejar en un bloque try/catch

HERENCIA Y EXCEPCIONES.

Ejemplo de extensión de una excepción:

```
public class PersonalizadaException extends Exception { 1 inheritance
    public PersonalizadaException(String mensaje) {
        super(mensaje);
    }
}
```

```
public class NoZeroIdException extends PersonalizadaException {
    public NoZeroIdException(String mensaje) {
        super(mensaje);
    }
}
```

HERENCIA Y EXCEPCIONES. Su uso en una clase:

```
public interface CrudRepositorio{ 1 implementation
    Cliente porId(int id) throws PersonalizadaException; 1 implementat
}

public class ClienteRepositorio implements CrudRepositorio{

    public Cliente porId(int id) throws PersonalizadaException {
        if (id == 0) {
            throw new NoZeroIdException("id no puede ser cero");
            // lanza una excepcion hija de la declarada en el throws
        }
    }
}
```

MALAS PRÁCTICAS AL MANEJAR EXCEPCIONES

=> Ignorar las excepciones o capturarlas sin hacer nada.

=> Capturar excepciones demasiado generales.

=> Usar excepciones para controlar el flujo del programa (condicionales o iteraciones). Es ineficiente y hace el código más difícil de leer y mantener. Las excepciones están diseñadas para manejar errores, no para controlar el flujo de ejecución en condiciones normales.



MALAS PRÁCTICAS AL MANEJAR EXCEPCIONES

=> Aunque es técnicamente posible, generalmente **no se recomienda capturar y manejar objetos de tipo Error** porque representan problemas graves como fallas de la JVM que una aplicación no debería intentar manejar.

=> **Lanzar excepciones no comprobadas.**

=> **No cerrar los recursos.**



BUENAS PRÁCTICAS AL MANEJAR EXCEPCIONES

=> Capturar las excepciones más específicas antes de las más generales.

Por ejemplo:



```
try {  
    // código que puede lanzar varias excepciones  
} catch (FileNotFoundException e){  
    // manejo específico para archivos no encontrados  
} catch (IOException e) {  
    // manejo para otras IOException  
} catch (Exception e) {  
    // manejo general para cualquier otra excepción  
}
```

BUENAS PRÁCTICAS AL MANEJAR EXCEPCIONES

- => Documentar las excepciones.
- => Usar excepciones comprobadas.
- => Lanzar excepciones específicas.



BUENAS PRÁCTICAS AL MANEJAR EXCEPCIONES

Cerrar los recursos adecuadamente (puede hacerse en el bloque finally, que se ejecuta independientemente de si se lanzó una excepción o no).



Una forma adecuada de hacerlo



```
FileReader reader = null;
try{
    reader = new FileReader( fileName: "archivo.txt");
    // leer archivo
} catch (IOException e) {
    // manejo de excepción
} finally{
    if (reader != null){
        try {
            reader.close();
        } catch (IOException e) {
            // manejo de excepción al cerrar
        }
    }
}
```

BUENAS PRÁCTICAS AL MANEJAR EXCEPCIONES

=> Utilizar el constructor que acepta una causa es útil para el encadenamiento de excepciones, donde un throwable es causado por otro. Esto es especialmente útil en situaciones donde se captura una excepción y se quiere lanzar una diferente sin perder el rastreo original.



=> Al crear excepciones personalizadas, `extendException` generalmente es suficiente.